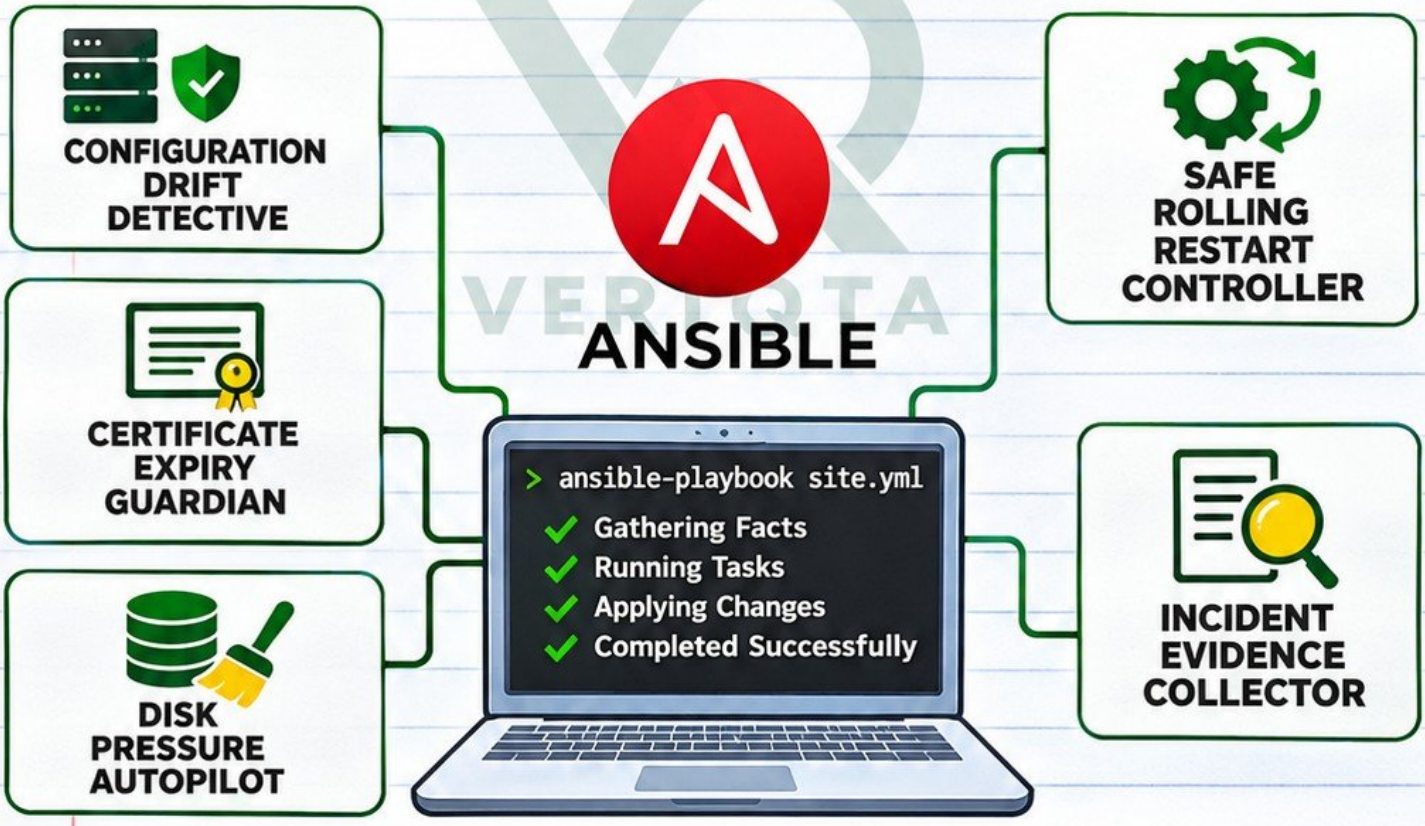
VERIQTA

5 Ansible Playbooks That Automate Your Busywork



- Real-World Automation for Operations, Reliability, and Speed
- 5 Production-Ready Playbooks with Code and Use Cases
- Save Time, Reduce Errors, and Keep Systems Healthy

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

Five Playbooks That Replace Hours of Operational Busywork

The Automation Problem Nobody Measures.

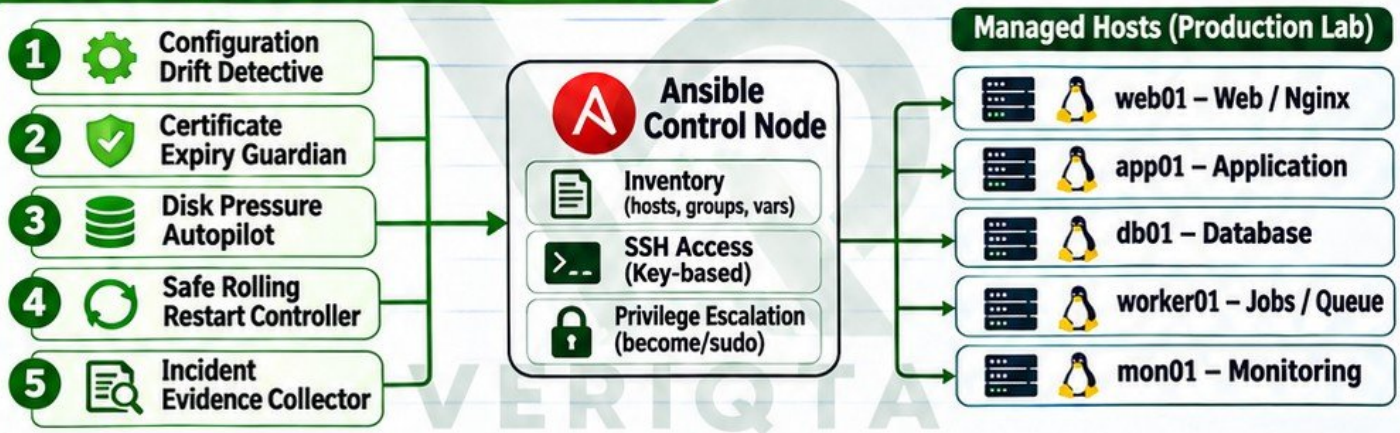
The Hidden Costs

- Repeated SSH sessions.
- Manual, error-prone checks.
- Inconsistent fixes across servers.
- Missing audit evidence.
- Hours lost on repetitive tasks.

The Fix: Automated Playbooks

- Standardize operational tasks.
- Reduce human error and drift.
- Provide consistent, auditable changes.
- Free up time for real engineering work.

Five-Playbook Architecture Map



The Automation Decision Flow



Shared Safety Controls (Use in Every Playbook)

- **Check Mode** – Test changes without applying.
- **Idempotency** – Safe to run multiple times.
- **Limits** – Control how many hosts change at once.
- **Approvals** – Require human approval for risky changes.
- **Rollback** – Return to the last known good state.

Production Lab: Five Linux Hosts

Host	Role	Purpose
web01	Web	Nginx / reverse proxy / TLS
app01	Application	Runs the business app (API or web)
db01	Database	Database service (e.g., PostgreSQL/MySQL)
worker01	Worker	Background jobs, queues, scheduled tasks
mon01	Monitoring	Observability (Prometheus, Grafana, logs)

Engineering Challenge

Define which operations may run automatically and which must require human approval, and justify your decision.

VERIQTA

The Emergency Fix That Became an Invisible Production Risk

Scenario: An engineer changes an Nginx configuration during an outage. Weeks later, the server behaves differently from every other production node.

What to Detect (Configuration Drift)

- **Nginx configuration** (e.g., /etc/nginx/).
- **SSH settings** (e.g., /etc/ssh/sshd_config).
- **systemd units** (e.g., /etc/systemd/system/).
- **Application files** (e.g., /opt/app/config/).

Approved vs. Observed Configuration

Approved (Baseline)

- ✓ From Git (version-controlled)
- ✓ Known good and tested
- ✓ Expected across all hosts

↔

Observed (Actual)

- ✗ Changed during outage
- ✗ Different from baseline
- ✗ May be missing or modified

Key Drift Signals to Check

- **File checksum** (e.g., SHA256) – has the content changed?
- **Ownership** (user:group) – is it still correct?
- **Permissions** (e.g., 644, 600) – has access been altered?
- **Provenance** – when, by whom, and how it changed (e.g., Git, manual, or incident fix?).

The Risk of a Blind Restore

- Can overwrite a valid emergency fix.
- May reintroduce the original outage.
- Can remove temporary but necessary changes (e.g., higher timeouts, custom limits).
- Ignores the context and reason for the change.

Deliverable

A drift-detection playbook that reports differences without changing the hosts.

What the Playbook Should Produce

- ✓ A clear list of files that differ from the approved baseline.
- ✓ Checksum, ownership, and permission changes.
- ✓ A per-host drift report (e.g., table or JSON).
- ✓ No changes made to any configuration (read-only).

Build a Drift Scanner That Explains What Changed

Key Ansible Building Blocks

ansible.builtin.stat
Get file metadata and checksums (e.g., SHA256).

ansible.builtin.slurp
Read file content (controlled inspection and comparison).

ansible.builtin.template
Render the approved configuration for comparison.

How the Drift Scanner Works

1. Collect file details (stat).
2. Read file content (slurp).
3. Render approved version (template).
4. Compare and detect changes.
5. Generate a structured JSON report.

Advanced Capabilities

- **Structured drift reports in JSON** (easy to read and integrate).
- **Host-specific exceptions** (e.g., allow temporary changes).
- **Environment-specific baselines** (dev, staging, production).
- **Secret-safe reporting** (mask sensitive values like passwords, tokens, and keys).

Example: Drift Report (JSON)

JSON

```
{
  "host": "web02",
  "file": "/etc/nginx/nginx.conf",
  "changed": true,
  "reason": "checksum_mismatch",
  "expected_sha256": "a9f4e2...",
  "observed_sha256": "d7c1b3...",
  "details": "Configuration differs
   from approved baseline"
}
```

Lab: Test the Drift Scanner

Deliberately modify three production-like hosts and generate a drift report identifying the affected files and hosts.

Lab Steps:

1. Set a known-good baseline.
2. Make controlled changes on 3 hosts (e.g., Nginx, SSH, systemd).
3. Run the drift scanner playbook.
4. Review the JSON report and affected hosts.

Repair Configuration Drift Without Recreating the Incident

Goal

Fix configuration drift safely, validate the change, and update the host without introducing a new outage.

Controlled Remediation Workflow

Detect (Drift Report) → Validate (Test Config) → Backup (Current Config) → Apply (If Valid) → Rollback (If Failed)

Remediation Build Blocks

- Separate detection from remediation.
- Validate Nginx with nginx -t before reload.
- Back up the previous configuration (versioned).
- Use handlers only when validated changes occur.
- Require explicit approval for protected files.
- Restore the previous configuration when validation fails.

Key Playbook Flow

```
1  - name: Deploy and validate Nginx config
2    block:
3      - name: Backup current config
         copy: src=/etc/nginx/nginx.conf
               dest=/etc/nginx/nginx.conf.bak
4      - name: Validate config
         command: nginx -t
5      - name: Reload Nginx (only if valid)
6        service: name=nginx state=reloaded
```

Handlers (Run Only on Valid Changes)

```
1  handlers:
2    - name: reload nginx
3      service: name=nginx state=reloaded
```

Failure Injection (Test Safety)

- Introduce an invalid Nginx directive (e.g., invalid_param on;).
- Run the remediation playbook.
- Verify that nginx -t fails.
- Confirm the playbook does not reload Nginx.
- Ensure the previous working configuration remains active.

Expected Outcome

- Drift is remediated safely and predictably.
- Only validated changes are applied.
- A backup of the previous configuration is kept.
- Protected files require explicit approval.
- Invalid changes are rejected and rolled back without downtime.

Prove Drift Automation Is Safe to Run Repeatedly

Objective

Demonstrate that the drift playbook is predictable, repeatable, and safe for production use.

Key Assurance Areas

- Idempotency verification.
- Check mode limitations and real validation.
- Controlled canary execution with serial.
- GitHub Actions integration for scheduled checks.
- Evidence reports and change-review artifacts.

Idempotency (Safe and Repeatable)

- Running the playbook multiple times produces the same state.
- No unnecessary changes on subsequent runs.
- Use idempotent modules (e.g., template, copy, file, systemd).

Check Mode vs. Real Validation

- Check mode (--check) shows planned changes only.
- It does not validate service behavior (e.g., Nginx config).
- Follow with real validation (e.g., nginx -t, service status).

Controlled Canary with Serial

- Roll out to a small batch first (e.g., serial: 1).
- Validate changes and service health.
- Proceed to the rest only if the canary is successful.

GitHub Actions (Scheduled Drift Check)

- Run the drift playbook on a schedule (e.g., daily or weekly).
- Use a dedicated workflow (YAML) to execute checks and store reports.
- Keep artifacts for audit and change review.

Example: Production Run Evidence

```
$ ansible-playbook drift.yml
  RUN 1 (Detection + Remediation)
  changed=3   ok=12   failed=0
$ ansible-playbook drift.yml
  RUN 2 (Repeat Run)
  changed=0   ok=15   failed=0
```

Result

- First run fixed the detected drift (changed=3).
- Second run made no changes (changed=0).
- The playbook is idempotent, stable, and safe to run repeatedly.

Production Acceptance

Run the playbook twice and prove that the second run makes no unnecessary changes.

VERIQTA

The Certificate That Expired on a Server Nobody Monitored

Scenario: A production API fails because an internal TLS certificate expired. External monitoring never detected the risk.

Certificate Types in Production

External Certificate
Public-facing (e.g., APIs, websites) signed by a trusted CA.

Internal Certificate
Used within your infrastructure (e.g., microservices, internal APIs).

Certificate Details to Check

- Expiration date (NotAfter).
- SAN coverage (all required domains).
- Issuer (CA and trust chain).
- Chain validation (full certificate chain).

Where Certificates Are Used

Nginx
(TLS termination for web traffic).

Internal APIs
(Service-to-service encryption).

Service Endpoints
(e.g., app, gateway, metrics, admin).

Inventory Certificate Paths and Ownership

Component	Typical Path	Owner
Nginx	/etc/nginx/ssl/*.crt	root
API Service	/opt/app/certs/*.pem	appuser
System/Host	/etc/pki/tls/certs/*.crt	root
Custom Service	/srv/service/certs/	svcuser

Certificate Health Report (Example)

HOST	SERVICE	EXPIRES_IN	STATUS
web01	nginx	42 days	OK
api01	api	5 days	WARNING
svc01	internal	EXPIRED	CRITICAL

Key Takeaways

- Track both internal and external certificates.
- Verify expiration, SANs, issuer, and chain.
- Cover Nginx, APIs, and all service endpoints.
- Maintain an approved inventory with ownership.

Deliverable: A certificate discovery and expiration-reporting playbook that finds all certificates, validates them, and reports at-risk and expired certificates.

Detect Certificate Risk Before the Expiration Date

Implementation (Ansible)

- Use community.crypto.x509_certificate_info.
- Discover certificates from approved paths (e.g., /etc/ssl, /etc/nginx/ssl).
- Apply thresholds for 30, 14, and 7 days.
- Generate host-level certificate health reports.
- Detect missing files, unreadable certificates, and invalid metadata.

Certificate Risk Thresholds

> 30 Days	Healthy — no action needed.
14 – 30 Days	Warning — plan rotation.
≤ 7 Days	Critical — rotate immediately.

Certificate Health Checks

- Certificate file exists and is readable.
- Valid certificate metadata (dates, issuer, SANs).
- Correct certificate chain (full chain verification).
- Ownership and permissions are correct.
- Report per host with status and days to expire.

Example: Ansible Task

```
- name: Get certificate details
  community.crypto.x509_certificate_info:
    path: /etc/nginx/ssl/server.crt
    register: cert_info
```

Sample Report (Per Host)

Host	Certificate	Expires In	Status
web01	/etc/nginx/ssl.crt	42 days	OK
api01	/etc/api/tls.crt	12 days	WARNING
svc01	/etc/ssl/old.crt	5 days	CRITICAL

Lab: Test the Playbook

1. Create a short-lived test certificate (e.g., valid for 5 days).
2. Run the certificate discovery playbook.
3. Generate the expiration report.
4. Verify that the report flags the certificate as CRITICAL.

Key Takeaways

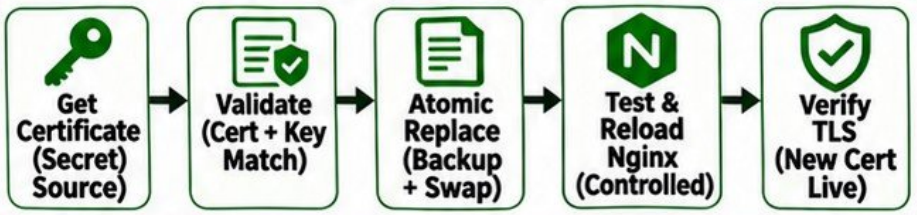
- Automate certificate discovery and risk detection.
- Use clear thresholds (30/14/7 days).
- Identify missing, unreadable, and invalid certificates.
- Produce a host-level, actionable expiration report.

Rotate Certificates Without Breaking Production Connections

Goal

Rotate TLS certificates safely and automatically without causing downtime or dropping active connections.

End-to-End Rotation Flow



Key Implementation Steps

1. Retrieve the certificate and key from an approved secret source (e.g., Vault, AWS Secrets Manager).
2. Handle the private key with no_log and restrictive permissions (e.g., 600, owned by root).
3. Verify the certificate and private key match (modulus/fingerprint).
4. Replace certificates atomically (backup, then move new files).
5. Validate the Nginx configuration (nginx -t) before reload.
6. Reload Nginx in a controlled way (only when validation succeeds).
7. Verify the new certificate with a TLS handshake (e.g., openssl s_client).

Example: Key Tasks (Ansible)

YAML

```yaml
- name: Deploy and reload certificate
  hosts: web
  vars:
    cert_dir: /etc/nginx/ssl
  tasks:
    - name: Copy certificate and key
      copy:
        src: "{{ item.src }}"
        dest: "{{ item.dest }}"
        mode: "{{ item.mode }}"
      loop:
        - { src: cert.pem, dest: "{{ cert_dir }}/cert.pem", mode: '0644' }
        - { src: key.pem, dest: "{{ cert_dir }}/privkey.pem", mode: '0600' }
    - name: Validate Nginx config
      command: nginx -t
    - name: Reload Nginx (if valid)
      service:
        name: nginx
        state: reloaded
```

Verification (Post-Rotation)

- Confirm the new certificate is served.
- Verify the certificate and key match.
- Check expiry date, issuer, and SANs.
- Ensure the service remains available (no downtime).



Failure Injection Test

1. Supply a mismatched certificate and key.
2. Run the playbook.
3. Confirm the deployment is rejected and the existing certificate remains active.

Result

- Certificates are rotated safely, validated, and reloaded.
- On invalid or mismatched inputs, the playbook stops and preserves the current working certificate.

Build a Certificate Rotation Audit Trail

Goal

Maintain a clear, verifiable record of certificate rotations, outcomes, and recovery actions.

End-to-End Audit Trail Flow

Capture Old Cert (Fingerprint) → Rotate Certificate (Safe) → Validate & Confirm (New Cert) → Record Outcome (Success/ Failure) → Alert & Retain Evidence

Key Evidence to Capture

- Old and new certificate fingerprints (SHA-256).
- Rotation success and failure records with timestamps.
- Pre-expiration alerts (e.g., 30, 14, 7 days).
- Recovery actions after a failed reload.
- Services that require a restart instead of a reload.

Example: Rotation Evidence

Rotation Audit Report

```
Service          : nginx
Host             : web01
Old Fingerprint  : 3A:8F:9D:...::2C
New Fingerprint  : 7B:E1:4D:...:9A
Result           : SUCCESS
Action           : reload (nginx)
Timestamp        : 2026-11-09 14:32:10Z
```

Recovery After a Failed Reload

1. Detect failed config reload (e.g., nginx -t error).
2. Restore the previous certificate and key from backup.
3. Reload or restart the service.
4. Verify service health and TLS handshake.

Reload vs. Restart

Service Type	Action	Example
Supports Reload	Reload (no downtime)	Nginx, HAProxy
Requires Restart	Restart (brief downtime)	App services (e.g., Gunicorn, Java, .NET)

Alerting Before Expiration

- Send alerts at 30, 14, and 7 days.
- Include host, service, expiry date, and SANs.
- Integrate with email, Slack, or other alert tools.

Production Acceptance

- The new certificate is served (fingerprint matches).
- The service remains reachable (e.g., HTTPS 200).
- Rotation is recorded with clear evidence.
- The private key never appears in logs (use no_log).

Result

A reliable, auditable, and recoverable certificate rotation process with clear evidence and no key exposure.

VERIQTA

The Server Is 95% Full, but Deleting Logs Could Destroy the Investigation

Scenario: A critical application host approaches disk exhaustion while an incident is being investigated.

Key Concepts to Understand

Filesystem Utilization
How much disk space is used (e.g., 95% full).

Inode Exhaustion
Running out of file indexes (inodes), even when disk space still appears available.

Common Space Consumers

- Log files (e.g., /var/log/*).
- Container artifacts (e.g., /var/lib/docker/*).
- Caches (e.g., package, app, proxy).
- Temporary files (e.g., /tmp, /var/tmp).

Why rm -rf Is Not a Recovery Strategy

- It can delete critical logs and evidence.
- It may remove required application data.
- It increases the risk of service instability or outage.

Safe vs. Protected Targets

Safer to Clean (with Validation)	Protected (Do Not Delete)
Old, rotated logs (e.g., *.log.1).	Current incident logs.
Expired cache files.	Application data and configs.
Unused container images.	Audit, security, and system logs.
Temporary files older than a set period.	Any evidence related to the ongoing investigation.

Disk-Pressure Diagnosis (Commands)

```
$ df -h                          # Filesystem usage
$ df -i                          # Inode usage
$ du -sh /var/log/* | sort -h    # Log size
$ docker system df               # Container usage
$ du -xh / | sort -rh | head -10 # Largest items
```

Deliverable

A disk-pressure diagnosis playbook that analyzes disk and inode usage, identifies the top space consumers, and recommends safe actions before execution.

Outcome

- Find the real cause of the disk pressure.
- Recommend safe, evidence-preserving cleanup options without deleting critical investigation data.

Build a Policy-Based Disk Recovery Engine

Goal

Automatically recover disk space using clear policies, safety controls, and a dry-run preview before deletion.

Implementation (Ansible)

- Use ansible.builtin.setup for filesystem facts.
- Check disk and inode utilization.
- Apply policy-based cleanup (allowlist, age, size, and file type).
- Enforce protected paths and minimum free-space targets.

Key Policy Controls

- **Allowlisted cleanup directories** (e.g., /var/log, /tmp/cache).
- **Age condition** (e.g., older than 7 or 30 days).
- **Size condition** (e.g., larger than 50 MB).
- **File-type condition** (e.g., .log, .tmp, .cache).
- **Minimum free-space target** (e.g., keep at least 20% free).
- **Protected paths** (e.g., /var/log/incident, /etc, /var/lib).

Example: Policy Variables

YAML

```yaml
disk_cleanup:
  allowlist:
    - /var/log
    - /tmp
    - /var/cache
  file_types: [".log", ".tmp", ".cache"]
  older_than_days: 30
  min_size_mb: 50
  min_free_space_percent: 20
  protected_paths:
    - /var/log/incident
    - /etc
```

Dry-Run Report (Preview)

Path	Size	Age	Action
/var/log/app.log	520 MB	45 days	Would Remove
/tmp/build/	1.2 GB	10 days	Would Remove
/var/log/incident.log	300 MB	2 days	Keep (Protected)

Lab: Test the Engine

1. Generate disposable test files (logs, tmp, cache).
2. Run the playbook in dry-run mode.
3. Review the report to confirm only approved files are selected.
4. Execute the cleanup and verify disk space is recovered and protected files remain.

Outcome

- A safe, policy-based disk recovery engine that identifies, previews, and cleans only approved files.
- Disk space is recovered while preserving investigation data and critical system files.

Recover Capacity Without Hiding the Root Cause

Goal

Free up disk space safely, maintain investigative evidence, and address the underlying cause of continuous disk growth.

Recovery Workflow

Assess Usage (Disk/Inode) → Archive / Rotate Logs → Cleanup Approved Temp Files → Recheck Capacity (After Each Step) → Stop if Unsafe (or Escalate)

Archive vs. Delete Decisions

Archive (Keep)

- Important logs and evidence.
- Required for incident analysis.
- Keep for a defined retention period (e.g., 30–90 days), then purge.

Delete (Safe)

- Approved temporary files (e.g., /tmp, cache).
- Old, rotated, or disposable files.
- Non-critical artifacts (e.g., build outputs, expired caches).

Log Rotation (Example)

```
/etc/logrotate.d/app:
/var/log/app/*.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
}
```



Recovery Steps (with Checks)

1. Check current disk and inode usage (e.g., df -h, df -i).
2. Archive or rotate application logs (e.g., logrotate).
3. Clean approved temporary files (e.g., /tmp, cache).
4. Recheck capacity after each stage.
5. Stop automation if utilization is still unsafe (e.g., > 90% or below target free space).

Escalation: Engineering Problem

- Identify persistent growth (e.g., large logs, runaway processes, or application bugs).
- Escalate to the engineering team with evidence (disk trends, top directories, file sizes).
- Treat recurring disk pressure as a system / application issue, not just ops cleanup.

Failure Injection Test

1. Create a large, protected file (e.g., /var/log/important.log).
2. Run the playbook to recover space.
3. Verify the protected file is not deleted.
4. Confirm the playbook reports “skipped – protected path.”

Key Takeaways

- Recover space safely and deliberately.
- Preserve evidence and important data.
- Verify capacity after each action.
- Stop if still unsafe and escalate the root cause.

Result

Disk space is restored safely, critical data is preserved, and persistent growth is escalated for a long-term fix.

VERIQTA

The Restart That Turned One Unhealthy Server Into a Full Outage

Scenario

An operator restarts every application server simultaneously, disconnecting active users and causing a full outage.

Key Concepts

- Load balancer target health and traffic routing.
- Connection draining (graceful termination).
- Application readiness vs. process status.
- Restart sequencing (one or few at a time).
- Minimum healthy capacity (keep required instances online).
- Why systemctl is-active is not enough (process can be running but app may not be ready).

Rolling Restart Flow

1. Check current LB health and healthy instance count.
2. Set the instance to drain (remove from rotation).
3. Wait for active connections to finish (connection draining).
4. Restart the application/service.
5. Verify application readiness (health check endpoint).
6. Re-add the instance to the LB and repeat for the next server.

Process vs. Application Readiness

Check	What It Tells You
systemctl is-active	The service process is running (active), but not a guarantee the application is ready.
Health endpoint (e.g., /health,/ready)	The application is actually ready to serve traffic (dependencies loaded, no errors).
LB target health (e.g., ALB / Nginx)	The instance is healthy and can receive production traffic.

Minimum Healthy Capacity

- Maintain a defined minimum (e.g., ≥ 50% or N–1).
- Restart one instance at a time (or a small batch).
- Ensure the service remains available throughout the restart.

Deliverable

A rolling restart playbook with explicit availability safeguards, including draining, readiness checks, sequencing, and minimum healthy capacity.

Result

- Servers are restarted safely, one at a time.
- The application remains available with no outage and no dropped user sessions.

Build a Restart Controller That Understands Service Health

Goal

Automate a safe, controlled service restart with readiness checks, bounded rollout, and clear failure handling.

Implementation (Ansible)

- Use serial for bounded rollout batches.
- Run pre_tasks for preflight checks.
- Use ansible.bulitin.uri for readiness probes.
- Use ansible.builtin.systemd_service for controlled restarts.
- Use wait_for and until for recovery verification.
- Enable any_errors_fatal and set failure thresholds.

Restart Flow

1. Preflight: verify LB, service, and health endpoint.
2. Restart: one node at a time (serial).
3. Wait: confirm the service is ready and healthy.
4. Proceed: move to the next node.
5. Abort: stop immediately if checks fail (any_errors_fatal).

Example: Playbook Snippet

YAML

```yaml
- hosts: app_nodes
  serial: 1
  any_errors_fatal: true
  pre_tasks:
    - name: Preflight health check
      uri:
        url: "http://{{ inventory_hostname }}:8080/health"
        method: GET
        status_code: 200
  tasks:
    - name: Restart application
      systemd_service:
        name: myapp
        state: restarted
    - name: Wait for readiness
      uri:
        url: "http://{{ inventory_hostname }}:8080/health"
        status_code: 200
      register: health
      until: health.status == 200
      retries: 12
      delay: 10
```

Lab: 3-Node Rolling Restart

- Restart three application nodes (one at a time).
- Continuously check a test endpoint (e.g., /health).
- Verify the service stays available throughout.

Verification Checks

Check	Expected Result
Readiness probe (uri)	HTTP 200 (service ready)
systemd_service	Service restarted (active)
wait_for / until	Returns success within the timeout
serial: 1	Only one node restarts at a time

Production Safeguards

- Bounded rollout (serial).
- Preflight and post-restart health checks.
- Automatic rollback/abort on repeated fails.
- Configurable failure thresholds and retries.

Outcome

- A reliable restart controller that restarts services safely, verifies health, and prevents a full outage.
- Proven in a 3-node rolling restart with continuous health checks.

Integrate Load Balancer Draining and Recovery

Goal

Safely restart application nodes by integrating AWS ALB draining, health verification, and rollback controls.

Key Concepts

- AWS ALB target deregistration and re-registration.
- Wait for connection draining (deregistration delay).
- Verify spare capacity before removing a node.
- Restart only after traffic has drained.
- Confirm the target is healthy before continuing.
- Use AWS collections with least-privilege IAM permissions.

Rolling Restart Workflow

1. Check ALB target health and total healthy capacity.
2. Deregister the target (allow connection draining).
3. Wait until the target is fully drained (no active connections).
4. Restart the application/service.
5. Re-register the target with the ALB.
6. Confirm target is healthy, then proceed to the next node.

Ansible (AWS ALB) Example

YAML

```yaml
- name: Deregister target from ALB
  amazon.aws.elb_target:
    state: absent
    target_group_arn: "{{ tg_arn }}"
    target_id: "{{ inventory_hostname }}"
- name: Wait for connection draining
  pause:
    seconds: "{{ drain_seconds }}"
- name: Restart application
  ansible.builtin.systemd_service:
    name: myapp
    state: restarted
- name: Re-register and wait for healthy
  amazon.aws.elb_target:
    state: present
    target_group_arn: "{{ tg_arn }}"
    target_id: "{{ inventory_hostname }}"
```

Least-Privilege IAM (Example)

- elasticloadbalancing:DeregisterTargets
- elasticloadbalancing:RegisterTargets
- elasticloadbalancing:DescribeTargetHealth
- elasticloadbalancing:DescribeTarget Groups

Failure Injection Test

1. Make one node fail readiness (e.g., return HTTP 503).
2. Run the rollout playbook.
3. Confirm the deployment stops on the failed node (any_errors_fatal).
4. Verify other nodes are not restarted.

Verification Checklist

- Target is deregistered and draining completes.
- Service restart succeeds.
- Target is re-registered and shows “healthy.”
- Rollback/stop occurs if health check fails.
- Spare capacity is maintained throughout.

Outcome

- A safe, ALB-integrated rolling restart with traffic draining, health verification, and automatic stop on failure.
- No capacity loss and no widespread outage.

Add Canary Releases, Abort Conditions, and Recovery

Goal

Roll out changes safely using a canary, enforce quality gates, automatically abort on failure, and roll back to a known-good version.

Key Implementations

- Canary-first execution (one or small subset).
- Latency and error-rate gates (SLO thresholds).
- Rollback to a known-good version (if gates fail).
- Automatic halt on failed health checks.
- Operator-readable rollout summaries (status, metrics).
- Evidence that the service stayed available throughout.

Canary Workflow

Deploy Canary	Update one node (or a small percentage).
↓	
Monitor Gates	Check latency, error rate, and health.
↓	
Continue Rollout	Proceed to remaining nodes if gates pass.
↓	
Abort & Rollback	Stop rollout and revert to the last known-good version if gates fail.

Example: Canary with Gates

YAML

```yaml
- hosts: app_nodes
  serial: 1
  vars:
    canary: true
    max_error_rate: 0.02   # ≤ 2%
    max_latency_ms: 500    # ≤ 500 ms
  tasks:
    - name: Deploy application version
      ansible.builtin.systemd_service:
        name: myapp
        state: restarted

    - name: Check health endpoint
      ansible.builtin.uri:
        url: "http://{{ inventory_hostname }}:8080/health"
        return_content: false
        status_code: 200
      register: health
      retries: 12
      delay: 10
      until: health.status == 200
```

Quality Gates (Example)

Metric	Threshold	Action
Error Rate	≤ 2%	Abort and rollback
Latency (p95)	≤ 500 ms	Abort and rollback
Health Check	HTTP 200	Continue rollout
Availability	≥ 99.9%	Proceed to next node

Rollback & Recovery

- Revert to the last known-good version.
- Restore service on the canary node.
- Confirm health and availability.
- Keep the remaining nodes unchanged.
- Generate a clear rollout summary (success/failed, metrics, and actions taken).

Lab / Proof

- Deploy a canary with an intentionally failing version (e.g., high error rate).
- Verify the rollout stops immediately.
- Confirm other nodes are not restarted and the service stays available.

Outcome

- A controlled, canary-driven rollout with clear abort conditions, rollback, and proven service availability.
- A failed canary prevents the remaining nodes from restarting.

The Incident Evidence That Disappeared During Troubleshooting

Scenario

Engineers restart services and rotate logs before collecting evidence. The original failure becomes impossible to reconstruct.

Key Takeaways

- Preserve evidence before any remediation.
- Capture system, service, and kernel state.
- Collect logs, metrics, ports, and config changes.
- Avoid unnecessary changes to incident hosts.
- Handle sensitive data in collected artifacts.

Evidence to Collect

System Time	Current time, timezone, and NTP status.
Boot ID	Unique boot identifier (ID of this boot session).
Service State	Status, failures, and last restart time.
Kernel Events	System and kernel messages (related to the failure).
Journal Logs	System and application logs (journalctl).
Resource Metrics	CPU, memory, disk, and network usage at the time.
Listening Ports	Open ports and the processes using them.
Config Changes	Recent configuration and package changes.

Read-Only Evidence Commands

CMD

```
# System time and timezone
timedatectl
# Boot ID
cat /proc/sys/kernel/random/boot_id
# Service state (example)
systemctl status <service>
# Kernel and system logs
journalctl -k -n 200
journalctl -u <service> -n 200
# Resource metrics (snapshot)
top -b -n 1 | head -n 20
free -h
df -h
# Listening ports
ss -tulpn
# Recent config and package changes
sudo ausearch -k config_change 2>/dev/null || true
rpm -qa --last| |head -n 20
```

Preservation Rules

- Collect evidence before any restart, log rotation, or configuration change.
- Use read-only commands (no changes to the host).
- Take a copy of critical logs and configs (e.g., /var/log and /etc) to a secure, separate location.
- Record a clear timeline (what, when, by whom).

Handling Sensitive Data

- Review artifacts for secrets (tokens, keys, passwords, PII).
- Mask or redact sensitive information before sharing.
- Store evidence securely with restricted access.
- Follow your organization's retention and privacy policy.

Deliverable

- A read-focused incident evidence collection playbook that preserves key system, service, and configuration data without altering the affected host.

Build a Multi-Host Incident Snapshot Collector

Goal

Automatically collect a consistent, read-only incident snapshot from multiple hosts, organize it per host, and preserve it for analysis.

Key Implementations

- ansible.builtin.command for approved read-only diagnostics.
- ansible.builtin.service_facts to capture service states.
- ansible.builtin.fetch to collect selected evidence files.
- Collect journalctl, ss, df, and free outputs.
- Create timestamped per-host evidence directories.
- Generate checksums and a collection manifest.

Evidence to Collect

Service Facts	All services and their running/failed state.
Journal Logs	Relevant system and application logs (journalctl).
Network Sockets	Listening ports and active connections (ss).
Filesystem	Disk usage and filesystem state (df -h).
Memory	Memory usage and availability (free -h).
Config/Logs	Selected logs and config files (read-only).

Playbook Snippet (Ansible)

YAML

```yaml
- hosts: app_nodes
  gather_facts: false
  tasks:
    - name: Collect service facts
      ansible.builtin.service_facts:

    - name: Run read-only diagnostics
      ansible.builtin.command: "{{ item }}"
      register: diag_out
      loop:
        - journalctl -u myapp -n 200 --no-pager
        - ss -tulpn
        - df -h
        - free -h
    - name: Fetch evidence files
      ansible.builtin.fetch:
        src: /var/log/myapp.log
        dest: "evidence/{{ inventory_honstame }}/"
        flat: false
```

Lab: Multi-Host Simulation

1. Simulate a failing service on one node (e.g., stop myapp).
2. Run the snapshot collector across multiple hosts.
3. Verify per-host evidence directories with timestamps.
4. Confirm checksums and the collection manifest are generated.

Collection Output (Example)

OUTPUT

```
evidence/
├── app01_2026-11-09T10-30-00Z/
│   ├── service_facts.json
│   ├── journalctl_myapp.log
│   ├── ss.txt
│   ├── df.txt
│   ├── free.txt
│   └── checksums.sha256
├── app02_2026-11-09T10-30-00Z/
└── manifest.json
```

Outcome

- A repeatable multi-host evidence collector that captures service, system, and resource state for incident analysis.
- Proven by simulating a failing service and collecting consistent, timestamped evidence from multiple hosts.

Turn Raw Incident Data Into a Defensible Investigation Package

Goal

Organize, validate, and package incident evidence into a complete, secure, and shareable bundle for investigation, review, and handoff.

Key Takeaways

- Group evidence by host and UTC timestamp.
- Keep original evidence and create clear summaries.
- Identify and document any collection gaps.
- Redact sensitive data in derived reports.
- Preserve evidence integrity and control access.
- Deliver a reusable, self-contained incident package.

Evidence Structure

```
incident-2026-11-09/
  host-app01/
    2026-11-09T10-30-00Z/
      raw/ (original logs, outputs)
      summary/ (analysis, notes)
  host-app02/
  host-app03/
  manifest.json
  collection-report.md
  redacted-report.md
  checksums.sha256
```

Example: Evidence Manifest

JSON

```json
{
  "incident_id": "INC-2026-1109-001",
  "collected_utc": "2026-11-09T10:30:00Z",
  "hosts": [
    {
      "host": "app01",
      "status": "success",
      "files": 6,
      "checksum": "a8f2...e9d1"
    },
    {
      "host": "app02",
      "status": "success",
      "files": 6,
      "checksum": "b7c3...f0a9"
    }
  ],
  "gaps": [],
  "redaction": "applied",
  "access": "restricted"
}
```

Verification & Quality Checks

- Confirm all expected hosts and files are present.
- Validate checksums for collected artifacts.
- Review and document any failed steps or gaps.
- Ensure redaction removed sensitive data (e.g., tokens, passwords, PII).
- Apply proper access controls (e.g., owner-only, 750).

Incident Handoff Package

- Include: timeline, summary, key findings, and next steps.
- Provide both raw evidence and a redacted summary.
- Use a clear, consistent directory and naming format.
- Include the collection manifest and integrity checks.
- Package as a single, shareable archive (e.g., .tar.gz).
- Allow another engineer to analyze without server access.

Outcome

- A complete, defensible, and self-contained incident investigation package.
- Another engineer can review, validate, and continue the investigation without accessing the original servers.

Final Production Challenge
Five Playbooks, One Real Incident

Scenario

An application has intermittent 502 errors, a certificate is nearing expiration, disk utilization is increasing, and configuration drift exists across the fleet.

Objective

Diagnose, stabilize, and fully resolve the incident using five production-ready playbooks, in the right order, with evidence, validation, and a reusable repository.

Order of Operations

1. Collect incident evidence (before any changes).
2. Detect configuration drift and identify changes that matter.
3. Check certificate validity and plan a safe rotation.
4. Recover disk capacity (without destroying evidence).
5. Execute a controlled rolling restart (only when justified).
6. Verify application health and produce a final operational report.

The Five Playbooks

#	Playbook	Purpose
1	Incident Evidence Collector	Capture and preserve system, service, and application state.
2	Configuration Drift Checker	Compare desired vs. actual configuration and report differences.
3	Certificate Rotation	Check expiry, deploy new cert, and safely reload the service.
4	Disk Recovery Engine	Reclaim capacity using policy-based cleanup (safe and auditable).
5	Controlled Rolling Restart	Restart application nodes with health checks and rollback.

Validation & Testing

- Functional tests (no 502 errors).
- Certificate is valid with new expiry date.
- Disk utilization meets the target (e.g., < 80%).
- Configuration drift resolved or documented.
- Rollout completed with no downtime.
- Failure-injection evidence (e.g., force a 502 to prove detection and recovery).

Final Deliverables

- Five executable playbooks (YAML).
- Inventory and variables (e.g., hosts.ini, group_vars/).
- Validation tests and clear expected results.
- Failure-injection evidence (screenshots or logs).
- Rollback instructions for each playbook.
- A GitHub-ready repository (code, README, runbook).

Outcome

- A fully resolved, evidence-backed incident using five integrated playbooks.
- A reusable, production-grade repository ready for your portfolio.